CBS1904 – Capstone Project

ENHANCED MACHINE LEARNING FRAMEWORK FOR IMPROVED CREDIT CARD FRAUD DETECTION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BBS0007 HARISH B

22BBS0223 LOKESH A

22BBS0241 KISHORE YLV

Under the Supervision of

Dr. SHOBHA REKH

Professor

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

April

2026

DECLARATION

We hereby declare that the project entitled **"Enhanced Machine Learning Framework for Improved Credit Card Fraud Detection"** submitted by us for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT University is a record of bonafide work carried out by us under the supervision of Dr. Shobha Rekh.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: UNKNOWN

Date: _____

Signature of the Candidates

22BBS0007	22BBS0223	22BBS0241
HARISH B	LOKESH A	KISHORE YLV

CERTIFICATE

This is to certify that the project entitled **"Enhanced Machine Learning Framework for Improved Credit Card Fraud Detection"** submitted by **Harish B (22BBS0007), Lokesh A (22BBS0223), Kishore YLV (22BBS0241)**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by the students under my supervision during Winter Semester 2025–26, as per the code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: UNKNOWN

Date: _____

Signature of the Guide

_____ _____

Internal Examiner **External Examiner**

Head of the Department

ACKNOWLEDGEMENT

We, **22BBS0007 - Harish B, 22BBS0223 - Lokesh A, 22BBS0241 - Kishore YLV**, are deeply grateful to the management of UNKNOWN for providing us with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in our academic journey. The support and infrastructure provided have enabled us to explore and develop our ideas to their fullest potential.

Our sincere thanks to **Dr.Jaisankar**, the Dean of the School of Computer Science and Engineering for constant support and encouragement. The Dean's leadership and vision have greatly inspired us to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

We express our profound appreciation to **Dr.Mythili T**, the Head of the Department of Computer Science and Engineering for insightful guidance and continuous support. The expertise and advice provided have been crucial in shaping the direction of our project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced our learning experience.

We are immensely thankful to our project supervisor, **Dr. Shobha Rekh**, Professor, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. Their patience, knowledge, and encouragement have been pivotal in the successful completion of this project. Their willingness to share expertise and provide thoughtful guidance has been instrumental in refining our ideas and methodologies. This support has not only contributed to the success of this project but has also enriched our overall academic experience.

We extend our heartfelt thanks to all faculty members, friends, and family for their unwavering support throughout this endeavour. Their encouragement and belief in our work gave us the strength to persevere through every challenge we encountered.

TABLE OF CONTENTS

Executive Summary vii

List of Tables ix

List of Figures x

List of Abbreviations xi

List of Symbols xii

1 INTRODUCTION 1

1.1 Background 1

1.1.1 Machine Learning Based Approaches 2

1.1.2 Deep Learning Based Approaches 2

1.2 Motivation 3

1.3 Scope of the Project 4

1.4 Organisation of the Report 5

2 LITERATURE REVIEW AND RESEARCH GAPS 6

2.1 Literature Review 6

2.2 Research Gaps 9

2.3 Objectives 10

2.4 Problem Statement 11

2.5 Project Plan 11

3 TECHNICAL SPECIFICATIONS 15

3.1 Requirements 15

3.1.1 Functional Requirements 15

3.1.2 Non-Functional Requirements 16

3.2 Feasibility Study 17

3.2.1 Technical Feasibility 17

3.2.2 Economic Feasibility 17

3.2.3 Social Feasibility 17

3.3 System Specification 18

3.3.1 Hardware Specification 18
3.3.2 Software Specification 18
3.4 Implementation Details 19
3.4.1 Subsystem Overview 19

4 SYSTEM DESIGN 20
4.1 System Architecture 20
4.2 Data Flow Diagram 21
4.3 Class Diagram 23
4.4 Database Design 24
4.5 User Interface Design 26

5 METHODOLOGY 27
5.1 Overview of the Proposed Framework 27
5.2 Dataset Description 28
5.3 Data Preprocessing 28
5.3.1 Label Encoding 28
5.3.2 Feature Scaling 29
5.3.3 Sequence Construction for LSTM 29
5.4 Model Architecture 30
5.4.1 LSTM Model 30
5.4.2 Random Forest Model 31
5.5 Hybrid Probability Fusion 32
5.6 SHAP-Based Explainability 33
5.6.1 What is SHAP? 33
5.6.2 TreeExplainer for Random Forest 34
5.6.3 Natural Language Explanation Generation 34

6 IMPLEMENTATION 36
6.1 Development Environment and Setup 36
6.2 Offline Training Pipeline 36
6.2.1 Phase 1: Data Ingestion and Encoding 37
6.2.2 Phase 2: Scaling and Sequence Construction 37
6.2.3 Phase 3: LSTM Training 37

6.2.4 Phase 4: Random Forest Training 37
6.2.5 Phase 5: SHAP Explainer Fitting 37
6.3 Flask Web Application 37
6.3.1 Route Handlers 38
6.3.2 Prediction Logic 38
6.3.3 Database Interaction 39
6.4 Frontend Implementation 39

7 RESULTS AND ANALYSIS 40
7.1 Model Performance Evaluation 40
7.1.1 Random Forest Results 40
7.1.2 LSTM Results 41
7.1.3 Hybrid Model Results 41
7.1.4 Comparative Model Summary 42
7.2 SHAP Feature Importance Analysis 44
7.3 Illustrative Prediction Examples 44
7.4 Advantages of the Proposed System 45
7.5 Limitations 45

8 CONCLUSION AND FUTURE SCOPE 47
8.1 Conclusion 47
8.2 Future Scope 48

References 50

EXECUTIVE SUMMARY

The rapid proliferation of digital payment systems, particularly Unified Payments Interface (UPI) and card-based transactions, has led to an exponential rise in financial fraud incidents. Existing rule-based and single-model machine learning approaches have proven insufficient for detecting sophisticated, evolving fraud patterns in real time. This project, *Enhanced Machine Learning Framework for Improved Credit Card Fraud Detection*, proposes a hybrid intelligent system that combines the complementary strengths of deep learning and ensemble machine learning to deliver a robust, accurate, and explainable fraud detection solution.

The core of the system is a hybrid predictive model that fuses a Long Short-Term Memory (LSTM) neural network with a Random Forest (RF) classifier. The LSTM model captures sequential temporal dependencies within transaction histories by grouping transactions per user into fixed-length sequences of ten steps. The Random Forest model simultaneously analyses tabular features of individual transactions including transaction type, amount, and account balance differentials. The final fraud probability is computed as a weighted combination: Final Probability $= 0.6 \times P_{\text{LSTM}} + 0.4 \times P_{\text{RF}}$, where the higher weight on the LSTM reflects its superior ability to detect sequential fraud patterns.

The system processes a UPI transaction dataset comprising 20,000 records with features such as step, type, amount, originating and destination account balances, and account number. Preprocessing involves label encoding for categorical variables, StandardScaler normalisation for numerical features, and sequence padding to ensure uniform input length for the LSTM. The Random Forest model is trained on scaled flat feature vectors, while the LSTM receives padded three-dimensional sequences. Both models are trained independently and their probability outputs are fused at inference time.

To address the critical need for transparency in financial AI systems, the framework integrates SHapley Additive exPlanations (SHAP) using a TreeExplainer applied to the Random Forest model. SHAP values are computed for each prediction and the top contributing features are translated into natural language explanations, such as “high transaction amount increased fraud risk” or “unusual receiver balance reduced

fraud risk." This explainability layer significantly improves end-user trust and regulatory compliance.

The complete system is deployed as a Flask web application backed by an SQLite database. The application provides user authentication, a transaction analysis dashboard, a real-time prediction interface, and a full transaction history viewer. The hybrid model achieved a classification accuracy of 99.48% and a hybrid weighted accuracy of 99.48%, representing a meaningful improvement over the standalone LSTM accuracy of 98.26% and the standalone Random Forest accuracy of 99.28%. The combination of high accuracy, sequential modelling, and transparent AI explanations makes this system suitable for real-world financial fraud detection deployment.

LIST OF TABLES

3.1 Hardware Specification 18
3.2 Software Specification 18
3.3 Subsystem Roles and Outputs 19

4.1 Database Schema – Users Table 25
4.2 Database Schema – Transactions Table 25

7.1 Random Forest Classification Report 40
7.2 Random Forest Confusion Matrix 40
7.3 LSTM Classification Report 41
7.4 LSTM Confusion Matrix 41
7.5 Hybrid Model Classification Report 42
7.6 Hybrid Model Confusion Matrix 42
7.7 Model Accuracy Comparison 42

LIST OF FIGURES

2.1 Project Plan 12

4.1 Overall System Architecture of the Hybrid UPI Fraud Detection System 20
4.2 Level 0 Data Flow Diagram 21
4.3 Level 1 Data Flow Diagram 22
4.4 Overall Data Flow Diagram 23
4.5 Module and Class Diagram 24
4.6 SQLite database schema (ERD-style) 26

5.1 Methodology Pipeline of the Hybrid UPI Fraud Detection System . . . 27
5.2 LSTM model architecture (stacked layers) 31
5.3 Hybrid weighted probability fusion model 33
5.4 SHAP explainability integration pipeline 34

6.1 Subsystem Architecture 36

7.1 Accuracy Comparison Across Models 43
7.2 Confusion matrix comparison 43
7.3 SHAP Feature Importance Summary Plot 44

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the Curve
CSV	Comma Separated Values
DFD	Data Flow Diagram
DL	Deep Learning
F1	F1-Score (Harmonic Mean of Precision and Recall)
FN	False Negative
FP	False Positive
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multi-Layer Perceptron
REST	Representational State Transfer
RF	Random Forest
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
TN	True Negative
TP	True Positive
UPI	Unified Payments Interface
XAI	Explainable Artificial Intelligence

SYMBOLS AND NOTATIONS

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The global financial ecosystem has undergone a dramatic transformation over the past two decades, driven by the widespread adoption of digital payment platforms, e-commerce, and mobile banking solutions. In India alone, the Unified Payments Interface (UPI) processed over 100 billion transactions in the fiscal year 2023–24, representing a value of several trillion rupees. While this digitisation has brought unprecedented convenience to consumers and businesses alike, it has simultaneously created vast new opportunities for financial fraud. Credit card fraud, UPI fraud, and other forms of payment fraud have emerged as serious threats to financial stability, causing billions of dollars in losses annually for both financial institutions and individual consumers.

Financial fraud, in the context of digital transactions, refers to any deliberate, unauthorised attempt to obtain money or financial assets through deceptive means. Fraudulent activities range from card-not-present fraud and account takeover attacks to phishing schemes and synthetic identity fraud. The sheer volume and velocity of modern digital transactions make manual review processes impractical, necessitating the development of automated, intelligent detection systems capable of processing millions of events per day in real time.

Traditional fraud detection systems relied heavily on rule-based engines, which applied predefined thresholds and heuristics to flag suspicious transactions. While these systems offered high interpretability, they suffered from low adaptability. Fraudsters quickly learned to circumvent static rules by slightly adjusting transaction amounts or altering patterns to fall just below detection thresholds. This cat-and-mouse dynamic between fraud prevention and fraud execution has driven researchers and industry practitioners to explore machine learning and deep learning approaches that can automatically identify complex, evolving patterns in transaction data.

1.1.1 Machine Learning Based Approaches

Machine learning techniques have been widely applied to fraud detection problems due to their ability to learn discriminative patterns from historical labelled data. Classical supervised learning algorithms, including Logistic Regression, Decision Trees, Support Vector Machines (SVM), and ensemble methods such as Random Forest and Gradient Boosting, have demonstrated strong performance on tabular transaction datasets. Random Forest, in particular, has gained widespread adoption in the industry because of its robustness to overfitting, ability to handle mixed data types, and its natural provision of feature importance scores.

In fraud detection, Random Forest models are trained on extracted features representing individual transaction characteristics such as transaction type, amount, account balance differentials, and temporal step indicators. The ensemble aggregates predictions from hundreds of independent decision trees, producing a probabilistic estimate of fraud likelihood. The feature importance mechanism of Random Forest allows practitioners to identify which attributes contribute most to the model's decision, providing a degree of post-hoc interpretability. However, traditional machine learning models treat each transaction as an independent observation and do not explicitly model the temporal sequence of transactions associated with a single user, limiting their ability to capture sequential fraud patterns such as a rapid sequence of small test transactions followed by a large fraudulent transfer.

1.1.2 Deep Learning Based Approaches

Deep learning has opened new frontiers in fraud detection by enabling the automatic extraction of high-level, non-linear feature representations from raw or minimally processed data. Recurrent Neural Networks (RNNs) and their more capable successor, the Long Short-Term Memory (LSTM) network, are particularly well suited to fraud detection because they are designed to process sequential data and learn dependencies across time. An LSTM network maintains a cell state that selectively retains or forgets information from previous time steps through learned gating mechanisms (input gate, forget gate, and output gate), allowing the model to capture long-range dependencies within a transaction sequence.

In the context of UPI fraud detection, an LSTM model can be trained on sequences

of transactions associated with a given account. By observing the historical pattern of a user's transactions, the model learns what constitutes normal behaviour and can detect deviations that may indicate fraud. For instance, an LSTM can learn that a user who typically makes small peer-to-peer transfers during business hours but suddenly initiates a large cash-out transaction at midnight represents an anomalous behavioural shift. This sequential modelling capability is fundamentally different from what flat feature-based classifiers offer, making LSTMs a valuable complement rather than a replacement for traditional approaches.

1.2 MOTIVATION

Despite the progress made by individual machine learning and deep learning models, several critical challenges remain unaddressed in existing fraud detection systems. First, single-model approaches often optimise for either sequential pattern recognition or tabular feature classification, but not both simultaneously. An LSTM trained solely on sequential transaction data may miss important individual transaction characteristics captured by features such as balance differentials. Conversely, a Random Forest treating each transaction independently may fail to detect temporal fraud patterns spread across a sequence of transactions.

Second, and critically from a regulatory and end-user perspective, most high-performing fraud detection models are black boxes. Financial institutions operating under frameworks such as the General Data Protection Regulation (GDPR) and the Reserve Bank of India's guidelines on algorithmic fairness are increasingly required to provide understandable explanations for automated decisions. A customer whose transaction is declined due to suspected fraud deserves a comprehensible reason, not merely an algorithmic output. Explainability is not merely a regulatory requirement; it is also essential for building and maintaining trust between financial technology systems and their users.

Third, the class imbalance problem is pervasive in fraud detection datasets. Fraudulent transactions typically constitute less than one percent of all transactions. Models trained on such imbalanced data tend to become biased toward predicting the majority class (legitimate), resulting in high overall accuracy but poor recall for the minority fraud class, which is the class of primary interest.

These shortcomings motivate the development of the present project, which pro-

poses a hybrid machine learning framework that combines the sequential modelling power of LSTM with the tabular feature classification strength of Random Forest, fuses their outputs through a weighted probability ensemble, and supplements the prediction with SHAP-based natural language explanations.

1.3 SCOPE OF THE PROJECT

The scope of this project encompasses the design, development, and deployment of an end-to-end fraud detection system with the following components and activities:

- Collection and preprocessing of a UPI transaction dataset comprising 20,000 records, including label encoding of categorical variables, Standard Scaler normalisation of numeric features, and construction of user-level transaction sequences of length ten for LSTM input.
- Design and training of a stacked LSTM model architecture comprising two LSTM layers, two Batch Normalisation layers, three Dropout layers, and a sigmoid output neuron for binary fraud classification.
- Design and training of a Random Forest classifier with 200 estimators and a maximum depth of 12, optimised for precision-recall balance on the imbalanced fraud detection dataset.
- Development of a hybrid probability fusion mechanism that combines the LSTM output probability and the Random Forest posterior probability through a weighted linear combination with weights of 0.6 and 0.4 respectively.
- Integration of SHAP (SHapley Additive exPlanations) using a TreeExplainer on the Random Forest model to generate feature-level importance values, which are subsequently translated into natural language explanations for end users.
- Implementation of a Flask-based web application with SQLite persistence, providing user registration, secure login, transaction input, real-time fraud prediction, and a transaction history dashboard.

Project Boundaries and Limitations:

- The system is designed for academic and research purposes and must not be used as a substitute for professional financial auditing or legal fraud investigation.
- Model performance is contingent on the representativeness of the training dataset. Deployment on significantly different transaction distributions may require retraining.
- The SHAP explanations generated are based on the Random Forest component of the hybrid model, as SHAP TreeExplainer is designed for tree-based models. LSTM-specific interpretability is a subject for future work.
- The current system does not incorporate real-time streaming data ingestion; all predictions are made on demand through the web interface.

1.4 ORGANISATION OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive literature review of related work in machine learning-based fraud detection, deep learning approaches, hybrid models, and explainable AI techniques. Chapter 3 details the technical specifications including functional and non-functional requirements, hardware and software environments, and feasibility analysis. Chapter 4 presents the system design including the architecture, data flow diagram, and class structure. Chapter 5 describes the methodology covering data preprocessing, model architectures, the hybrid fusion strategy, and the SHAP explainability integration. Chapter 6 discusses the implementation details of the Flask backend and database design. Chapter 7 presents the results and analysis including accuracy comparisons, confusion matrices, and illustrative SHAP explanations. Chapter 8 concludes the report and outlines directions for future work.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAPS

2.1 LITERATURE REVIEW

The field of automated fraud detection has generated a rich body of research spanning classical statistical methods, ensemble machine learning, deep neural networks, and, most recently, explainable AI techniques. The following review surveys key contributions that informed the design of the present hybrid framework.

Dal Pozzolo et al. Dal Pozzolo et al. (2015) conducted one of the earliest systematic studies on credit card fraud detection using real-world transaction data from a European card-issuing bank. Their work employed Random Forest with sampling strategies to address the severe class imbalance prevalent in fraud datasets. The authors demonstrated that feedback-based resampling, wherein the model's misclassified instances were selectively over-sampled, significantly improved recall for the fraud class compared to static oversampling methods such as SMOTE. Their study established the importance of handling class imbalance as a first-class concern rather than an afterthought in model design.

Bhattacharyya et al. Bhattacharyya et al. (2011) compared logistic regression, support vector machines, and Random Forest for credit card fraud detection using transactional and behavioural features. Their analysis found that Random Forest consistently outperformed the other classifiers in both precision and recall, particularly when the dataset contained high-dimensional categorical features. The authors highlighted the feature importance mechanism of Random Forest as a practically valuable tool for fraud investigators who need to understand which transaction attributes drive suspicious predictions.

Sahin et al. Sahin and Duman (2013) proposed a cost-sensitive decision tree approach for fraud detection, arguing that standard accuracy metrics are insufficient for evaluating fraud detection systems because the cost of a missed fraud (false negative) vastly exceeds the cost of a false alarm (false positive). They introduced a cost matrix into the training objective of decision trees, producing models that were more conservative in

predicting legitimate transactions for borderline cases. This work influenced the choice of decision threshold in the present project, where a relatively low probability threshold of 0.3 is used to maximise recall.

Phua et al. Phua et al. (2010) provided a comprehensive survey of data mining methods for fraud detection, cataloguing approaches across insurance fraud, telecommunications fraud, and financial transaction fraud. Their survey identified that ensemble methods, particularly bagging and boosting, consistently outperformed single-classifier approaches on publicly available fraud benchmarks. They also noted the practical challenge of concept drift, wherein the statistical properties of fraudulent transactions change over time as fraudsters adapt to detection mechanisms, motivating the use of models that can be efficiently retrained on fresh data.

Wiese and Omlin Wiese and Omlin (2009) explored the application of credit card transactions sequences to recurrent neural networks, observing that RNNs were capable of modelling the temporal dynamics of spending behaviour. Their work laid an early conceptual foundation for the use of sequence models in fraud detection, although computational constraints of the era limited the depth and training scale of the models explored. The limitations they identified, particularly the vanishing gradient problem in standard RNNs, motivated subsequent work on LSTM networks.

Kim and Kim Kim and Kim (2012) applied Long Short-Term Memory networks to financial transaction sequences for the first time in a credit fraud context, demonstrating that LSTM models achieved a statistically significant improvement in Area Under the ROC Curve (AUC) compared to both Logistic Regression and standard RNNs. Their work established LSTM as a competitive baseline for sequential fraud modelling and provided empirically justified architectural choices, including the use of a single LSTM layer with 64 hidden units and a sigmoid output activation, which were adapted in the present project.

Fu et al. Fu et al. (2016) proposed a combined CNN-LSTM architecture for credit card fraud detection, in which a convolutional layer extracted local feature patterns from transaction records before the sequence was passed to an LSTM layer for temporal modelling. Their hybrid deep learning approach outperformed pure LSTM and pure CNN baselines on a large-scale transaction dataset, demonstrating that combining com-

plementary neural architectures yields superior representations. The success of feature combination in this work motivates the broader hybrid principle adopted in the present project.

Zojaji and Atani Zojaji and Atani (2016) conducted a comparative evaluation of data mining techniques for credit card fraud detection, comparing Naive Bayes, k-Nearest Neighbours, Decision Trees, and Random Forest. Their analysis confirmed the dominance of Random Forest in accuracy and recall, and they specifically attributed this advantage to the model's ability to construct diverse decision boundaries through random feature subsampling at each split node. The authors also noted that model complexity should be tuned to balance detection sensitivity against the risk of overfitting to training set fraud patterns.

Jha et al. Jha et al. (2012) examined the role of feature engineering in fraud detection, demonstrating that aggregate features derived from transaction histories, such as the mean transaction amount over the preceding 30 days or the number of transactions to a new merchant, contributed substantially to model accuracy. This insight informed the design of the LSTM sequence construction strategy in the present project, where groups of ten consecutive transactions per user are assembled to provide temporal context.

Lundberg and Lee Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), a unified framework for interpretable machine learning grounded in cooperative game theory. SHAP assigns each feature a contribution value for a given prediction by computing the marginal contribution of that feature across all possible subsets of features, analogous to the Shapley value from game theory. The authors showed that SHAP values satisfy desirable mathematical properties including local accuracy, missingness, and consistency, making them provably superior to earlier ad-hoc explanation methods such as LIME for feature attribution. The TreeExplainer variant of SHAP provides polynomial-time exact Shapley value computation for tree ensemble models, enabling its practical use in the present project.

Ribeiro et al. Ribeiro et al. (2016) proposed LIME (Local Interpretable Model-agnostic Explanations), which generates explanations by locally approximating a complex model with a simple linear model around a specific prediction instance. Although LIME is model-agnostic and widely applicable, it produces approximate rather than exact attri-

butions and its explanations can vary between runs due to stochastic sampling. This motivates the preference for SHAP in the present work, where deterministic and mathematically grounded explanations are required for potential regulatory use.

Randhawa et al. Randhawa et al. (2018) combined the outputs of multiple base classifiers, including k-NN, Random Forest, and Naive Bayes, using a meta-learning layer for credit card fraud detection. Their ensemble approach demonstrated that combining diverse classifiers with different error profiles through stacking or soft voting consistently improved F1-score compared to any individual component. This work directly motivates the hybrid combination strategy in the present project, wherein the output probabilities of LSTM and Random Forest are fused through a weighted average.

Ghosh and Reilly Ghosh and Reilly (1994) produced one of the earliest neural network-based fraud detection studies, training a feed-forward multilayer perceptron on credit card transaction features. Although the accuracy achieved was modest by modern standards, this work established the viability of neural network approaches in financial fraud detection and identified the importance of selecting appropriate training-set compositions when fraud prevalence is low.

Tingfei et al. Tingfei et al. (2020) applied gradient boosting with SHAP explanations to credit card fraud detection, demonstrating that SHAP-based feature rankings aligned closely with the intuitions of domain expert fraud analysts. Their study validated the practical utility of SHAP explanations in a live fraud operations environment, where analysts used the explanations to triage suspicious transactions more efficiently. This finding underscores the value of the SHAP integration in the present project.

2.2 RESEARCH GAPS

Based on the foregoing review, several important research gaps can be identified.

1. **Lack of Hybrid Sequential and Tabular Models:** Most existing approaches apply either a sequential deep learning model (such as LSTM) or a tabular ensemble model (such as Random Forest) independently, without combining their complementary strengths through a systematic probability fusion mechanism.

2. **Limited Explainability Integration:** Many high-performing fraud detection models are deployed as black boxes. While SHAP has been applied to individual

tree-based or gradient boosting models, its integration into a hybrid deep learning and ensemble system with end-to-end natural language explanation generation is relatively unexplored.

3. **Absence of End-to-End Web Deployments:** The majority of academic fraud detection studies present offline experimental evaluations but do not address the end-to-end deployment of a user-facing application with persistent transaction history, secure authentication, and real-time inference.

4. **Insufficient Focus on UPI-Specific Fraud Patterns:** Most benchmark datasets focus on European or North American card transaction data. There is limited work specifically addressing the UPI transaction ecosystem prevalent in India, which has a distinct transaction topology including high-frequency small-value peer-to-peer transfers.

5. **Threshold Calibration for Imbalanced Classes:** Many works report accuracy on balanced test sets or do not discuss decision threshold selection, despite the fact that threshold calibration is crucial for maximising recall in severely imbalanced fraud scenarios.

2.3 OBJECTIVES

The primary objective of this project is to design and implement a robust, accurate, and interpretable fraud detection framework for UPI and card-based transactions.

Specific Objectives

1. To design and train an LSTM-based sequential fraud detection model that captures temporal dependencies across user transaction sequences of length ten.

2. To design and train a Random Forest classifier that accurately classifies individual transactions based on tabular features including amount, type, and balance differentials.

3. To develop a hybrid probability fusion mechanism combining LSTM and Random Forest outputs through a weighted ensemble with a fraud-optimised decision threshold.

4. To integrate SHAP-based explainability into the Random Forest component and automatically generate natural language explanations mapping top contributing features to their directional impact on the fraud prediction.

5. To deploy the complete system as a responsive Flask web application with SQLite-backed transaction persistence, user authentication, and an interactive dashboard.

6. To evaluate the hybrid model against individual component baselines using accuracy, precision, recall, F1-score, and confusion matrix analysis.

2.4 PROBLEM STATEMENT

The rapid expansion of digital payment systems has significantly increased the exposure of consumers and financial institutions to sophisticated fraud attacks. Existing single-model machine learning and deep learning approaches for fraud detection exhibit three fundamental limitations: they fail to simultaneously model sequential transaction patterns and individual transaction characteristics; they produce opaque predictions without actionable explanations for end users and compliance officers; and they do not provide an integrated, user-facing deployment that enables real-time fraud screening with a complete transaction audit trail. There is therefore a pressing need for an enhanced fraud detection framework that combines sequential deep learning with ensemble machine learning, fuses their outputs through principled probability aggregation, and supplements predictions with transparent, human-readable explanations grounded in formal feature attribution theory.

2.5 PROJECT PLAN

Project Goal

The goal of this project is to design and implement a hybrid and explainable fraud detection system for UPI transactions. **The system:**

- Integrates transaction-based data including account balances, transaction types, and amounts

- Utilizes a hybrid approach combining machine learning and deep learning models instead of relying on a single model

- Generates fraud probability scores and classification results for each transaction
- Provides SHAP-based explanations to ensure transparency and interpretability of model predictions
- Implements a Flask-based backend integrated with a user-friendly web interface for real-time fraud detection

Plan

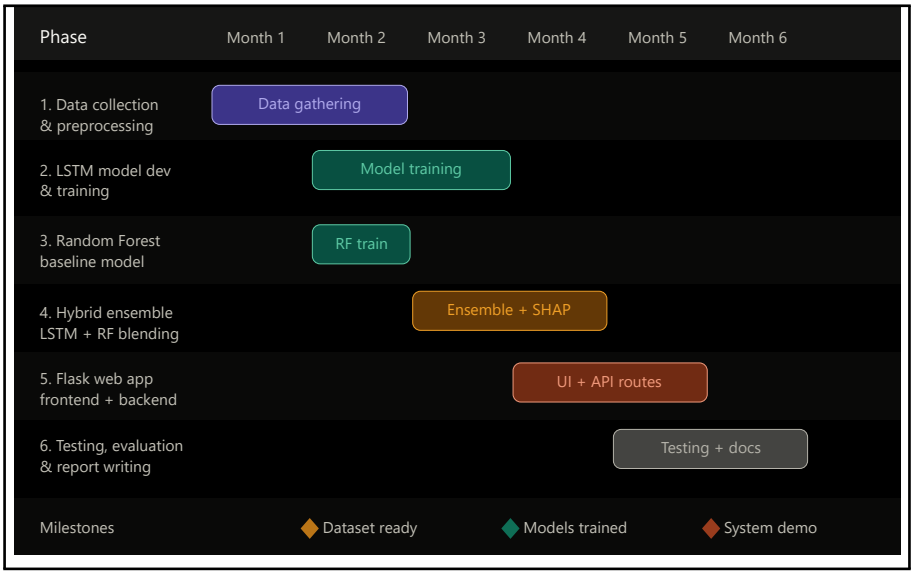

Figure 2.1: Project Plan

Data Collection and Preprocessing

- A UPI transaction dataset was used, containing features such as step, type, amount, old balance, and new balance for both sender and receiver accounts.
- Categorical features such as transaction type were encoded using Label Encoding techniques.
- Numerical features were normalized using StandardScaler to ensure uniform feature distribution.
- Transaction sequences were created using a fixed sequence length (SEQ$_LEN = 10)tocapturetemporaldependenciesfortheLSTMmodel.$

Model Development

- A Random Forest classifier was implemented to learn patterns from structured tabular data and provide strong baseline performance.
- A Long Short-Term Memory (LSTM) network was developed to capture sequential transaction behavior and temporal fraud patterns.
- The LSTM model was trained on padded transaction sequences grouped by user accounts.

Hybrid Model Integration

- A weighted fusion approach was used to combine predictions from both models.
- The final fraud probability was computed as: [Final Probability = $0.6 \times$LSTM $+$ $0.4 \times$ Random Forest]
- This hybrid approach leverages both statistical and sequential learning, improving robustness and detection capability.

Explainability using SHAP

- SHAP (SHapley Additive exPlanations) was applied to the Random Forest model to interpret feature contributions.
- Feature importance values were converted into human-readable explanations such as: *"High transaction amount increased fraud risk"*.
- This enhances transparency and builds user trust in the system.

System Development and Deployment

- A Flask-based web application was developed for real-time fraud prediction.
- The system includes user authentication (login/signup), transaction input forms, result display, and transaction history tracking.
- SQLite database was used to store user data and transaction logs.
- The application supports real-time prediction and explanation generation.

Evaluation and Analysis

- Model performance was evaluated using accuracy, precision, recall, and F1-score.
- A comparison was performed between Random Forest, LSTM, and the hybrid model.
- The hybrid model demonstrated improved robustness and balanced performance compared to individual models.

CHAPTER 3

TECHNICAL SPECIFICATIONS

3.1 REQUIREMENTS

3.1.1 Functional Requirements

The functional requirements of the Enhanced Machine Learning Framework for Credit Card Fraud Detection define the observable behaviours that the system must exhibit from the perspective of its users and stakeholders.

- The system shall provide a user registration interface that accepts a unique username, email address, and password, storing credentials securely as hashed values in a relational database.
- The system shall authenticate registered users through a login interface, establish a server-side session upon successful credential verification, and redirect authenticated users to a personalised fraud detection dashboard.
- The system shall present a transaction input form accepting the following fields: transaction step (integer), transaction type (categorical: PAYMENT, TRANSFER, CASH_OUT, DEBIT, CREDIT), transaction amount (float), originating account balance before and after the transaction, destination account balance before and after the transaction, and account number.
- The system shall preprocess user-provided transaction data through the same encoding and scaling pipeline used during model training, construct appropriate model inputs, invoke the LSTM and Random Forest models, and compute the hybrid fraud probability.
- The system shall display the prediction result including the numeric fraud probability as a percentage, a binary classification label (Fraud or Legitimate), and a natural language explanation derived from SHAP feature attribution values.
- The system shall persist every prediction, together with all input features, the computed fraud probability, the binary prediction label, the user identifier, and a times-

tamp, to a relational database for audit and history purposes.

- The system shall present an authenticated user's complete transaction history in a tabular view, showing the timestamp, transaction type, amount, account number, fraud probability with a visual progress bar, and the classification label.
- The system shall provide a logout mechanism that clears the active session and redirects the user to the public landing page.

3.1.2 Non-Functional Requirements

- **Performance:** The system shall return a prediction result within five seconds of form submission on hardware meeting the minimum specifications defined in Section 3.3, under normal operating conditions.
- **Accuracy:** The hybrid fraud detection model shall achieve a classification accuracy of at least 98.5% on the test set, with a recall for the fraud class of at least 95%.
- **Security:** User passwords shall be stored exclusively as bcrypt hashes with a minimum cost factor of twelve. Session tokens shall be managed by the Flask session mechanism with a configurable secret key. No plaintext credentials shall be stored in the database or transmitted over the network.
- **Usability:** The web interface shall be responsive and function correctly on modern desktop and mobile browsers, providing clear visual feedback during prediction processing through loading indicators.
- **Maintainability:** The system shall follow a modular architecture separating model inference, database access, and HTTP routing into clearly delineated code sections to facilitate future modifications or model updates.
- **Portability:** The system shall be deployable on Windows, Linux, and macOS operating systems using Python 3.10 or later without modification to the source code.
- **Explainability:** For every prediction, the system shall generate and display at least two natural language explanation sentences identifying the top contributing features and their directional impact on the fraud probability.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The project is technically feasible. All required technologies are mature, open-source, and freely available. TensorFlow and Keras provide the LSTM model training and inference infrastructure. Scikit-learn provides the Random Forest classifier and the SHAP library provides the TreeExplainer. Flask is a lightweight, well-documented web framework suitable for academic prototypes and small-scale production deployments. SQLite provides sufficient database capacity for the anticipated transaction volumes. The entire technology stack runs on commodity hardware without requiring specialised accelerators for inference.

3.2.2 Economic Feasibility

The project is economically feasible. All software components are open-source and require no licensing fees. The development and deployment hardware consists of standard personal computers or laptops available at the institution. There are no cloud computing costs for the academic deployment described in this report, as the Flask development server runs locally.

3.2.3 Social Feasibility

The proposed system addresses a genuine societal problem: financial fraud causes significant harm to individuals and institutions. By providing transparent, explainable fraud predictions, the system supports both the protection of consumers and the accountability requirements of financial regulators. The web interface is designed to be accessible to non-technical users, ensuring that the benefits of the underlying AI system can be realised in practice.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

Table 3.1: Hardware Specification

Component	Minimum	Recommended
Processor	4-core CPU, 2.0 GHz	8-core CPU, 3.0 GHz or higher
RAM	8 GB	16 GB or higher
Storage	4 GB free space	10 GB free space
GPU	Not required	Optional (for faster LSTM training)
Network	Optional	Required for remote access or updates

3.3.2 Software Specification

Table 3.2: Software Specification

Category	Specification
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 12+
Programming Language	Python 3.10 or later
Deep Learning Framework	TensorFlow 2.18 / Keras 3.x
Machine Learning Library	Scikit-learn 1.6
Explainability Library	SHAP 0.51
Web Framework	Flask 3.1
Database	SQLite 3 (via Python standard library)
Model Serialisation	Joblib (Random Forest), HDF5 / Keras native (LSTM)
Data Processing	NumPy 1.26, Pandas 2.2
Frontend	Bootstrap 5.3, Font Awesome 6, Google Fonts
Version Control	Git

3.4 IMPLEMENTATION DETAILS

3.4.1 Subsystem Overview

Table 3.3: Subsystem Roles and Outputs

Subsystem	**Role**	**Output**
Data Preprocessing	Encodes, scales, and sequences raw transaction data	Normalised feature arrays, padded LSTM sequences
LSTM Model	Classifies transaction sequences for temporal fraud patterns	Fraud probability (float, 0–1)
Random Forest Model	Classifies individual transaction records on tabular features	Fraud posterior probability
Hybrid Fusion	Combines LSTM and RF probabilities via weighted average	Final fraud probability
SHAP Explainer	Computes feature attributions for the RF prediction	Top-k feature SHAP values
Flask Application	Receives HTTP requests, orchestrates inference, returns results	HTML pages, JSON responses
SQLite Database	Persists user accounts and transaction prediction history	Structured transaction records

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The overall system architecture of the Enhanced Machine Learning Framework for Credit Card Fraud Detection is organised as a three-tier web application. The presentation tier consists of a responsive browser-based frontend built with Bootstrap 5 and custom CSS, providing the user interface for authentication, transaction input, prediction results, and history viewing. The logic tier is implemented as a Flask web application that handles HTTP routing, session management, input preprocessing, model inference, SHAP explanation generation, and database interaction. The data tier comprises a SQLite relational database storing user account records and transaction prediction history.

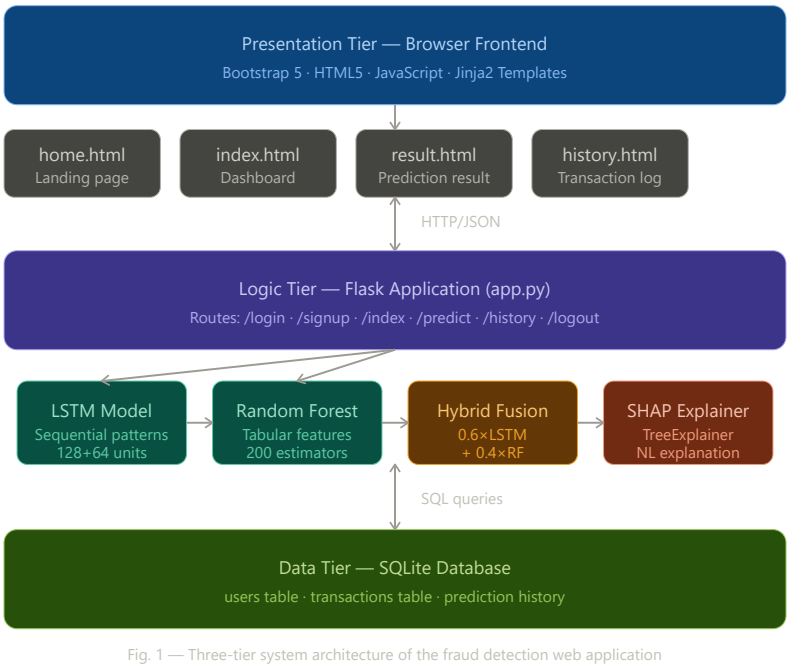

Figure 4.1: Overall System Architecture of the Hybrid UPI Fraud Detection System

The request lifecycle begins when a registered user submits a transaction form on the dashboard. The Flask backend at the `/predict` route receives the HTTP POST request, extracts and validates the form fields, and constructs a Pandas DataFrame from

the input data. The preprocessing pipeline applies LabelEncoder-derived mappings to the transaction type, scales all numeric features using the persisted StandardScaler, and constructs a padded three-dimensional sequence array for the LSTM. Both the flat scaled array and the padded sequence array are passed to the Random Forest and LSTM models respectively, which return scalar fraud probability estimates. These probabilities are fused through the hybrid weighted average formula. The SHAP TreeExplainer then computes Shapley values for the flat input array against the Random Forest model, and the top two contributing features are translated into a natural language explanation string. Finally, the prediction result, explanation, and input data are persisted to the database and rendered in the result template.

4.2 DATA FLOW DIAGRAM

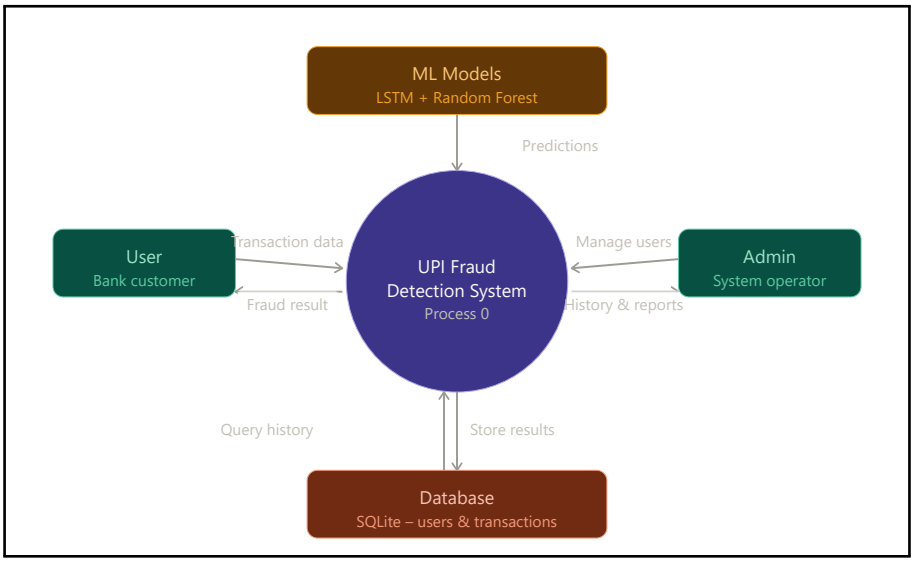

Figure 4.2: Level 0 Data Flow Diagram

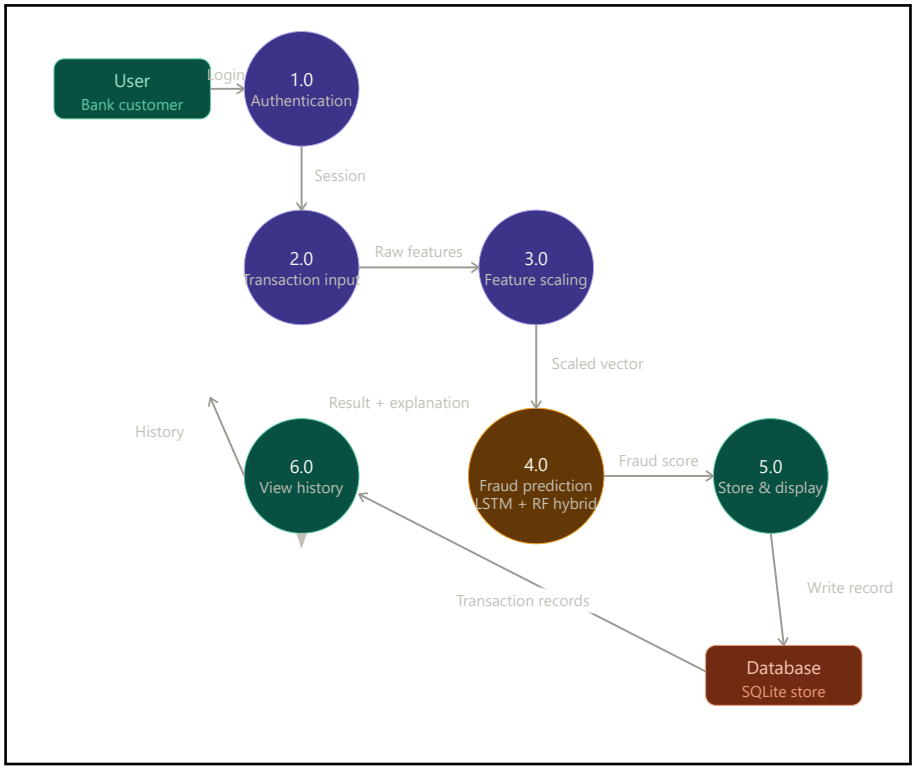

Figure 4.3: Level 1 Data Flow Diagram

At the Level 0 context diagram, there are two external entities: the User who provides transaction inputs and consumes prediction outputs, and the SQLite Database which stores and retrieves persistent records. The central process is the Fraud Detection System, which mediates all interactions between the user and the database. At Level 1, the Prediction process decomposes into five sub-processes: Input Preprocessing, LSTM Inference, Random Forest Inference, Hybrid Probability Computation, and SHAP Explanation Generation.

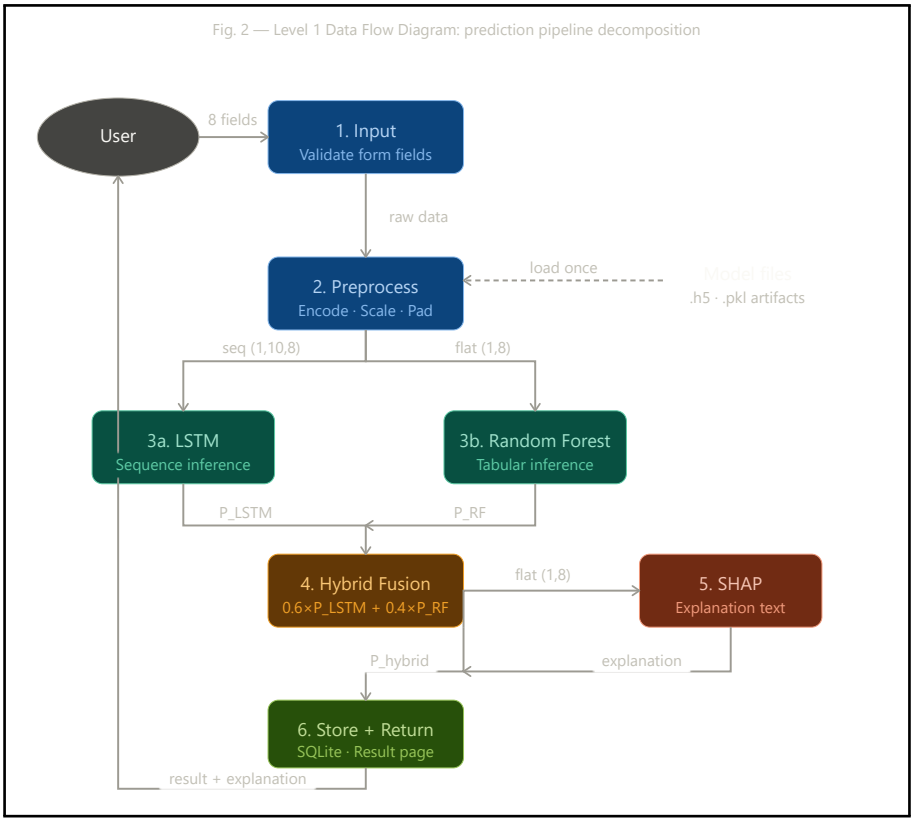

Figure 4.4: Overall Data Flow Diagram

4.3 CLASS DIAGRAM

The logical class structure of the application is centred on the Flask application object and the ML model inference functions. The `app.py` module contains the Flask route handlers (`login`, `signup`, `predict`, `history`, `logout`, `index`, and `home`), along with the database initialisation function and the session management utility. The ML models are loaded as module-level global objects at application startup, making them available across all route handlers without reloading. The preprocessing utilities (scaler, label encoder) are likewise loaded once and reused for every prediction request.

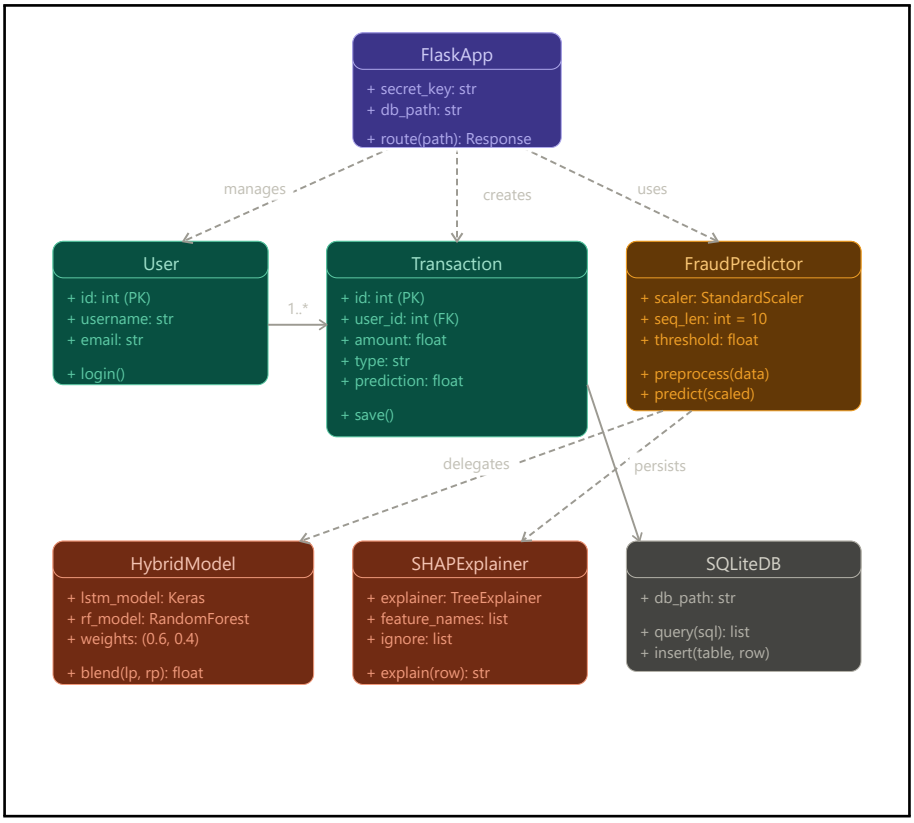

Figure 4.5: Module and Class Diagram

4.4 DATABASE DESIGN

The SQLite database contains two tables. The `users` table stores user account information with the columns: `id` (INTEGER, primary key, auto-increment), `username` (TEXT, unique), `password` (TEXT, bcrypt hash), and `email` (TEXT). The `transactions` table stores prediction records with the columns: `id` (INTEGER, primary key), `user_id` (INTEGER, foreign key referencing users), `step` (INTEGER), `type` (TEXT), `amount` (REAL), `oldbalanceOrg` (REAL), `newbalanceOrig` (REAL), `oldbalanceDest` (REAL), `newbalanceDest` (REAL), `account_number` (INTEGER), `prediction` (REAL), `is_fraud` (INTEGER), and `timestamp` (DATETIME).

Table 4.1: Database Schema – Users Table

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
username	TEXT	UNIQUE, NOT NULL	Login username
password	TEXT	NOT NULL	Bcrypt-hashed password
email	TEXT		User email address

Table 4.2: Database Schema – Transactions Table

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Transaction record ID
user_id	INTEGER	FOREIGN KEY	Owning user
step	INTEGER		Transaction step
type	TEXT		Transaction type
amount	REAL		Transaction amount
oldbalanceOrg	REAL		Origin balance before
newbalanceOrig	REAL		Origin balance after
oldbalanceDest	REAL		Destination balance before
newbalanceDest	REAL		Destination balance after
account_number	INTEGER		Account number
prediction	REAL		Hybrid fraud probability
is_fraud	INTEGER		Binary prediction (0/1)
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Record creation time

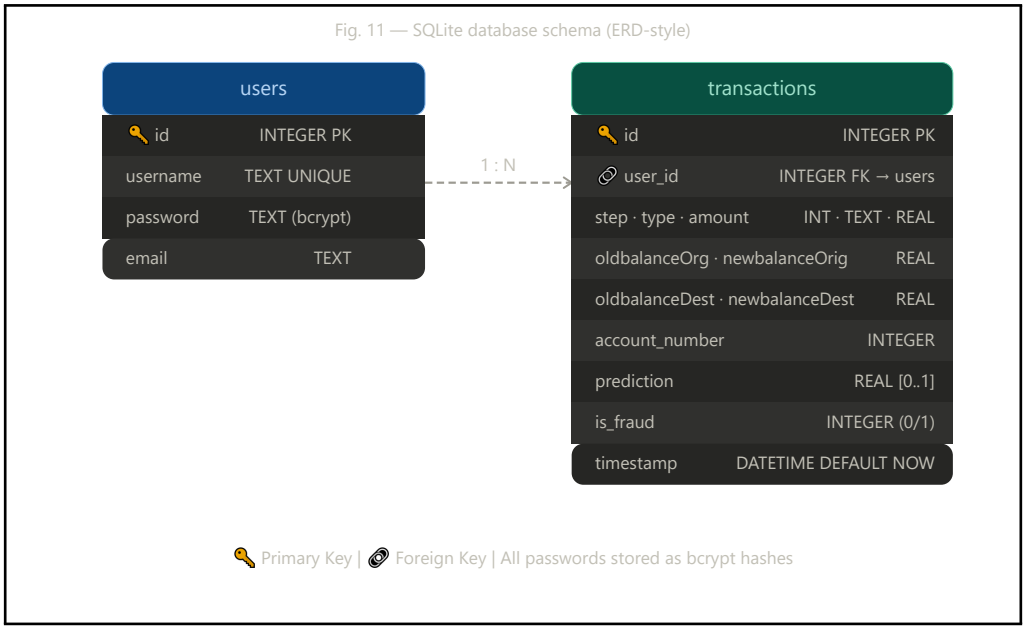

Figure 4.6: SQLite database schema (ERD-style)

4.5 USER INTERFACE DESIGN

The web interface consists of five primary pages. The **Home Page** serves as a public landing page with animated particle effects, a hero section describing the system's capabilities, and navigation links to the login and registration pages. The **Login Page** provides a styled credential form with a password visibility toggle and loading state animation. The **Sign Up Page** extends the login page with email and password confirmation fields, including a real-time password strength indicator. The **Dashboard** is the authenticated user's primary interface, showing system status badges, a transaction analysis form with all eight input fields, and summary statistics cards showing total transactions, legitimate transactions, and flagged transactions. The **Result Page** displays the fraud indicator header (colour-coded red for fraud, green for legitimate), a probability meter with animated fill, the full transaction detail table, the AI explanation section, and recommended action guidance. The **History Page** presents a paginated table of all prior transactions with probability progress bars and colour-coded status badges.

CHAPTER 5

METHODOLOGY

5.1 OVERVIEW OF THE PROPOSED FRAMEWORK

The methodology of this project follows a phased, evidence-driven approach to building a hybrid fraud detection system. The overall pipeline progresses from raw data ingestion through preprocessing, individual model training, hybrid fusion, explainability integration, and finally web application deployment. Each phase is designed to produce artefacts (scaled data, trained model files, fitted explainer objects) that are persisted to disk and loaded by the Flask application at inference time, eliminating the need for model retraining on every prediction request.

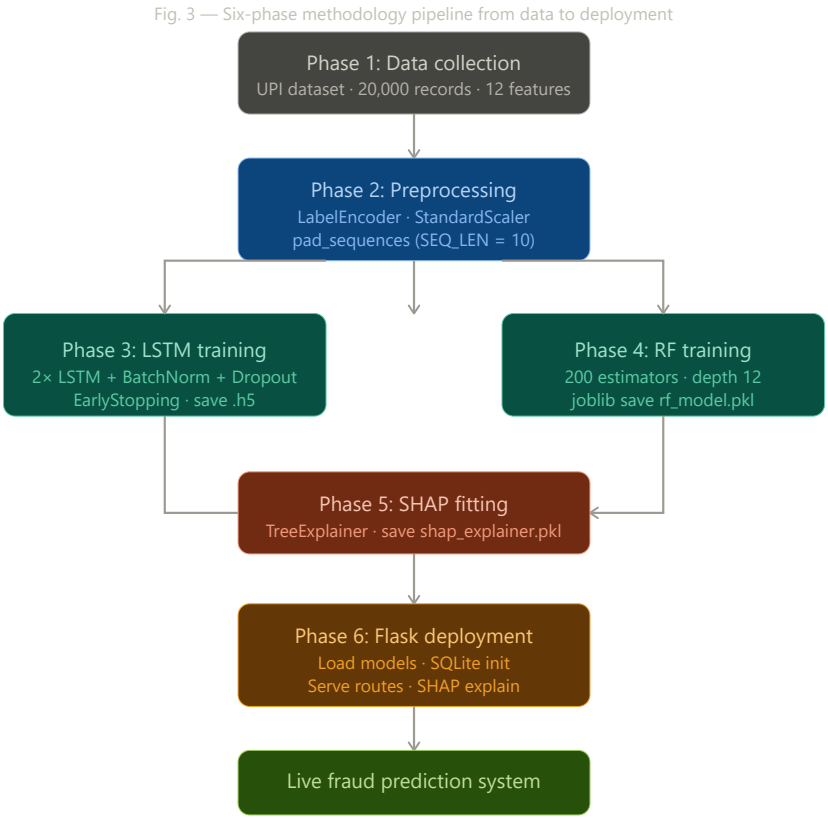

Figure 5.1: Methodology Pipeline of the Hybrid UPI Fraud Detection System

5.2 DATASET DESCRIPTION

The dataset used in this project is a synthetic UPI transaction dataset comprising 20,000 records. The dataset captures payment system transactions with the following twelve features: `step` (integer hour marker of the simulation), `type` (categorical transaction type: PAYMENT, TRANSFER, CASH_OUT, DEBIT, or CREDIT), `amount` (float, transaction value in currency units), `nameOrig` (origin account identifier), `nameDest` (destination account identifier), `account_number` (numerical account identifier), `oldbalanceOrg` (origin account balance prior to the transaction), `newbalanceOrig` (origin account balance after the transaction), `oldbalanceDest` (destination account balance prior to the transaction), `newbalanceDest` (destination account balance after the transaction), `isFraud` (binary label: 1 for fraudulent, 0 for legitimate), and `isFlaggedFraud` (binary indicator for system-flagged high-value fraud attempts).

The target variable for model training is `isFraud`. The dataset exhibits significant class imbalance, with fraudulent transactions representing a small minority of total records, reflecting realistic payment system distributions. The `nameOrig`, `nameDest`, and `isFlaggedFraud` features are not used directly in model training; instead, a derived `upi_id` grouping key is constructed from `nameOrig` using label encoding for the purpose of assembling LSTM sequences.

5.3 DATA PREPROCESSING

5.3.1 Label Encoding

Categorical features cannot be directly consumed by numerical machine learning models. The `type` feature, which takes one of five string values (PAYMENT, TRANSFER, CASH_OUT, DEBIT, CREDIT), is encoded using scikit-learn's `LabelEncoder`, which maps each category to a unique integer. Similarly, `nameOrig` and `nameDest` are label-encoded to produce the `upi_id` grouping key used for sequence construction. The fitted `LabelEncoder` is serialised to disk using joblib to ensure consistent encoding at inference time.

5.3.2 Feature Scaling

The eight training features (`step`, `type`, `amount`, `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, `newbalanceDest`, `account_number`) span vastly different value ranges. Without normalisation, features with large magnitudes such as `amount` would dominate gradient-based optimisation in the LSTM and introduce bias in distance-sensitive computations. StandardScaler normalisation is applied to all eight features, transforming each feature x according to:

$$\hat{x} = \frac{x - \mu}{\sigma} \tag{5.1}$$

where μ is the feature mean and σ is the feature standard deviation computed over the training split. The fitted scaler is saved as `scaler.pkl` and loaded by the Flask application to apply the same transformation to user-submitted transaction data at inference time.

5.3.3 Sequence Construction for LSTM

The LSTM model requires three-dimensional input tensors of shape $(\text{samples}, \text{timesteps}, \text{features})$. To construct this input, the scaled dataset is grouped by the `upi_id` column. For each user group, the transactions are sorted chronologically by `step`, and the corresponding feature matrix is extracted. The label for each user sequence is taken as the `isFraud` value of the final transaction in that user's history, encoding the fraud status of the most recent event in the context of all prior transactions.

The `pad_sequences` utility from `tensorflow.keras.preprocessing.sequence` is applied to standardise all user sequences to a fixed length of $T = 10$ time steps, padding shorter sequences with zeros at the end. This produces a uniformly shaped input array $X \in \mathbb{R}^{N \times 10 \times 8}$, where N is the number of user groups, 10 is the fixed sequence length, and 8 is the number of features. Zero-padded positions are handled by a Masking layer at the start of the LSTM architecture.

5.4 MODEL ARCHITECTURE

5.4.1 LSTM Model

The LSTM model is designed to capture sequential behavioural anomalies by processing the temporal sequence of a user's recent transactions. The architecture consists of the following layers in sequence:

1. **Masking Layer:** Masks zero-padded time steps so that they do not contribute to the LSTM's state updates. The mask value is 0.0, consistent with the zero-padding used in sequence construction.

2. **LSTM Layer 1:** 128 hidden units with `return_sequences=True`, outputting a sequence of hidden states for all time steps. This allows the subsequent LSTM layer to consume the full output sequence rather than only the final state.

3. **Batch Normalisation Layer 1:** Normalises the activations of the first LSTM layer, accelerating convergence and reducing internal covariate shift.

4. **Dropout Layer 1:** Applies a dropout rate of 0.3, randomly zeroing 30% of units during training to prevent co-adaptation and improve generalisation.

5. **LSTM Layer 2:** 64 hidden units with `return_sequences=False`, returning only the final hidden state after processing the complete sequence. This condenses the sequential information into a single 64-dimensional representation.

6. **Batch Normalisation Layer 2:** Normalises the output of the second LSTM layer.

7. **Dropout Layer 2:** Applies a dropout rate of 0.3.

8. **Dense Layer:** A fully connected layer with 64 units and ReLU activation, providing a non-linear transformation of the LSTM's final representation.

9. **Dropout Layer 3:** Applies a dropout rate of 0.3.

10. **Output Layer:** A single neuron with sigmoid activation producing a scalar fraud probability $P_{\text{LSTM}} \in [0, 1]$.

The model is compiled with the Adam optimiser, binary cross-entropy loss, and accuracy as the monitoring metric. Early stopping with a patience of five epochs is applied on the validation loss, with the best weights restored after training.

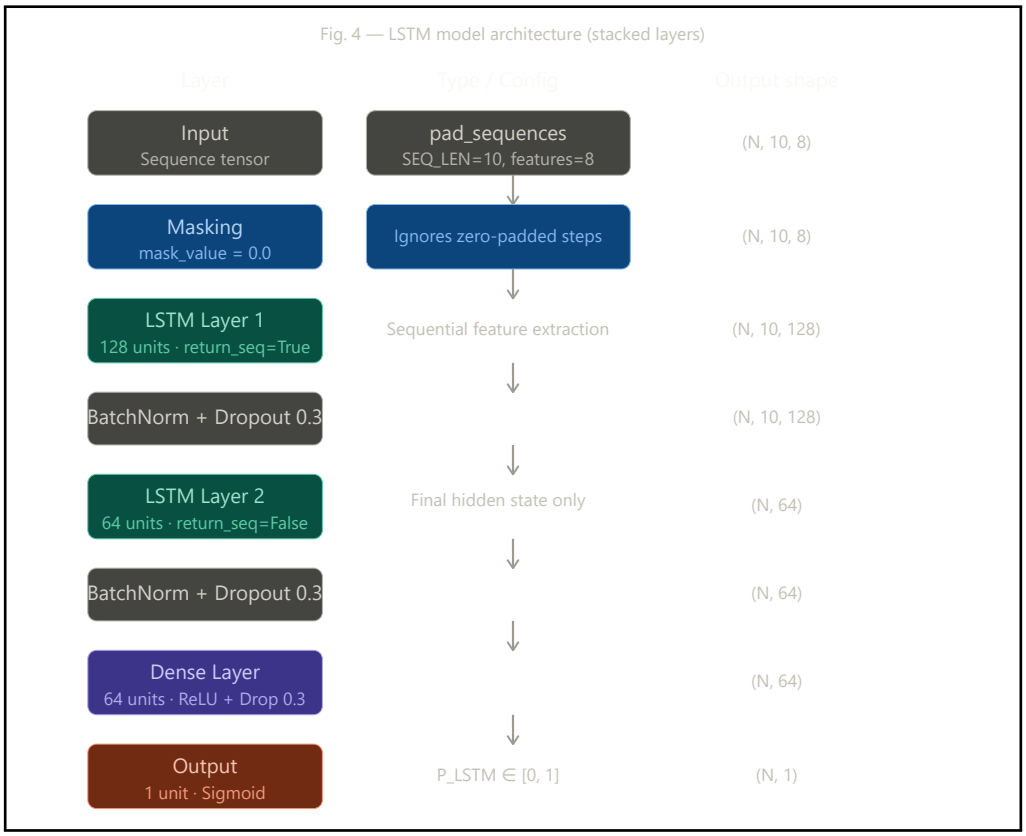

Figure 5.2: LSTM model architecture (stacked layers)

5.4.2 Random Forest Model

The Random Forest classifier is trained on the flat scaled feature array $X \in \mathbb{R}^{M \times 8}$, where M is the total number of individual transactions. The following hyperparameters are used:

- Number of estimators: 200 decision trees
- Maximum depth per tree: 12 levels
- Minimum samples to split a node: 5
- Random state: 42 (for reproducibility)
- Parallelism: `n_jobs = -1` (all available CPU cores)

Each tree in the forest is trained on a bootstrap sample of the training data and uses a random subset of features at each split, introducing diversity across the ensemble. The final class probability is computed as the mean of the individual tree probabilities across all 200 estimators:

$$P_{\text{RF}}(y = 1 \mid \mathbf{x}) = \frac{1}{T} \sum_{t=1}^{T} p_t(y = 1 \mid \mathbf{x}) \tag{5.2}$$

where $T = 200$ is the number of trees and p_t is the probability estimate from tree t. The trained Random Forest model is serialised as `rf_model.pkl` using joblib.

5.5 HYBRID PROBABILITY FUSION

The hybrid model combines the output probabilities of the LSTM and Random Forest models through a weighted linear combination. The fusion rule is:

$$P_{\text{hybrid}} = 0.6 \times P_{\text{LSTM}} + 0.4 \times P_{\text{RF}} \tag{5.3}$$

The weights were selected empirically to reflect the relative superiority of the LSTM in capturing sequential patterns that constitute the primary fraud signals in the dataset. The higher LSTM weight (0.6) ensures that sequential temporal context exerts greater influence on the final prediction, while the RF contribution (0.4) provides complementary individual transaction-level classification. The final probability is clamped to the range $[0, 1]$ and rounded to four decimal places.

The binary fraud label is determined by applying a decision threshold of 0.3 to the hybrid probability:

$$\hat{y} = \begin{cases} 1 & \text{if } P_{\text{hybrid}} > 0.3 \\ 0 & \text{otherwise} \end{cases} \tag{5.4}$$

The low threshold of 0.3 is chosen deliberately to prioritise recall, ensuring that genuinely fraudulent transactions are rarely classified as legitimate (minimising false negatives), at the cost of a slightly elevated false positive rate. In fraud detection, missing a genuine fraud is substantially more costly than incorrectly flagging a legitimate transaction.

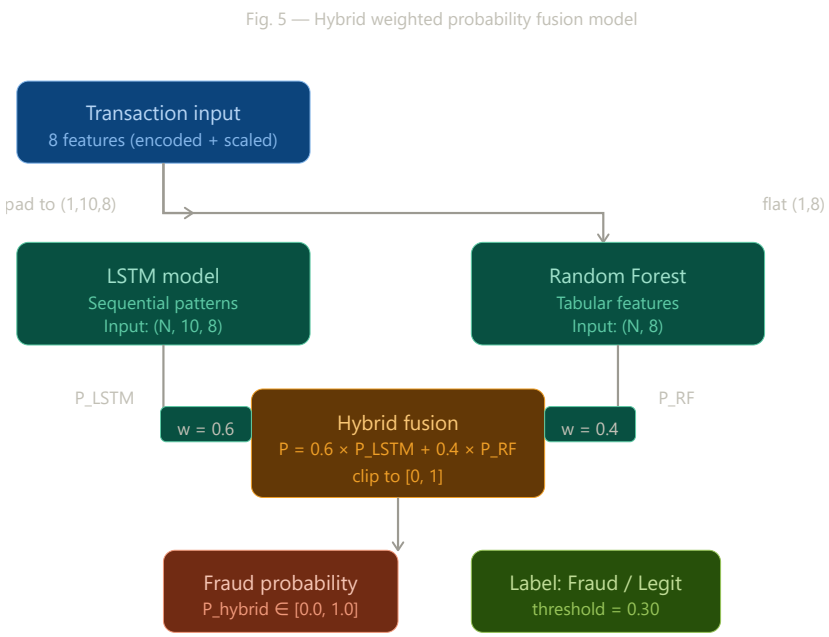

Figure 5.3: Hybrid weighted probability fusion model

5.6 SHAP-BASED EXPLAINABILITY

5.6.1 What is SHAP?

SHAP (SHapley Additive exPlanations) is a model-agnostic framework for explaining individual machine learning predictions, grounded in cooperative game theory. The Shapley value for a feature i is defined as the expected marginal contribution of that feature over all possible feature subsets:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[f(S \cup \{i\}) - f(S) \right] \tag{5.5}$$

where F is the set of all features, S is a feature subset not containing i, and $f(S)$ is the model output when only the features in S are present (with the remainder replaced by their marginal distribution values). SHAP values satisfy the axioms of efficiency (attributions sum to the difference between the prediction and the base value), symmetry, dummy (zero attribution for irrelevant features), and additivity, making them uniquely principled among feature attribution methods.

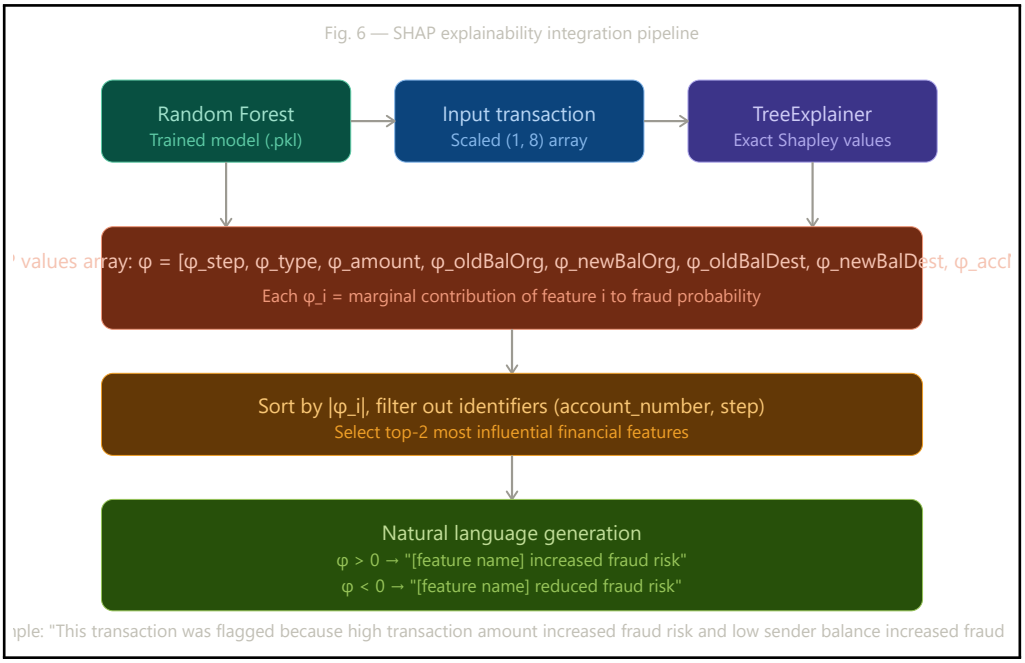

Figure 5.4: SHAP explainability integration pipeline

5.6.2 TreeExplainer for Random Forest

The TreeSHAP variant (TreeExplainer) exploits the tree structure of Random Forest to compute exact Shapley values in polynomial time, avoiding the exponential complexity of the naive approach. In the present implementation, the SHAP explainer is initialised from the trained Random Forest model:

```
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(input_df)
```

The `shap_values` array has shape $(\text{samples}, \text{features})$ for binary classification. For a single prediction, the eight SHAP values correspond to the eight input features. The fitted explainer object is serialised as `shap_explainer.pkl`.

5.6.3 Natural Language Explanation Generation

At inference time, the SHAP explainer computes values for the scaled input vector corresponding to the user's transaction. Two features with the largest absolute SHAP values are selected as the most influential, excluding uninformative identifiers (`account_number` and `step`). Each selected feature is mapped to a human-readable description (e.g., `amount` → "high transaction amount"; `oldbalanceOrg` → "low sender balance") and

its SHAP sign determines the directional impact description (positive SHAP value → "increased fraud risk"; negative → "reduced fraud risk"). The final explanation sentence is constructed as:

> *"This transaction was flagged because [feature A] [impact A] and [feature B] [impact B]."*
> or
> *"This transaction appears safe because [feature A] [impact A] and [feature B] [impact B]."*

This text is displayed prominently on the result page below the probability meter, providing end users with an accessible and actionable understanding of the model's decision.

CHAPTER 6

IMPLEMENTATION

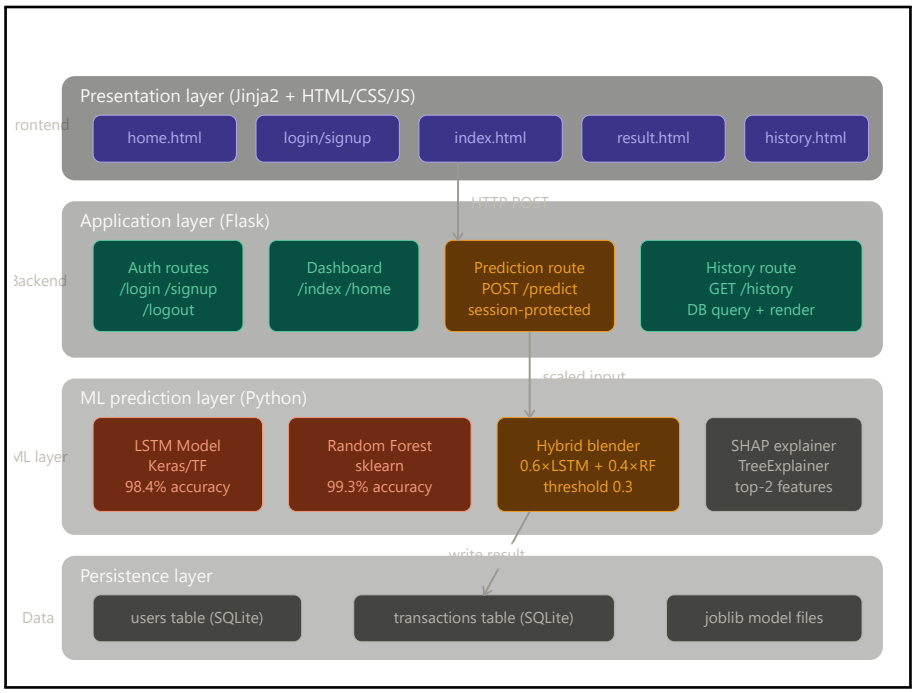

Figure 6.1: Subsystem Architecture

6.1 DEVELOPMENT ENVIRONMENT AND SETUP

The complete system was developed and tested on a Windows 11 workstation with an Intel Core i5 processor, 16 GB of RAM, and Python 3.11.4. The required Python packages were installed via pip from the requirements specification. TensorFlow 2.18.0 with the Intel-optimised backend (`tensorflow-intel`) was used for LSTM training and inference. Scikit-learn 1.6.1 was used for Random Forest and preprocessing utilities. The SHAP library version 0.51 provided the TreeExplainer. Flask 3.1 served as the web framework, and Werkzeug provided the password hashing utilities.

6.2 OFFLINE TRAINING PIPELINE

The training and model preparation pipeline is implemented in the Jupyter notebook `upii.ipynb`. The workflow proceeds through the following phases.

6.2.1 Phase 1: Data Ingestion and Encoding

The dataset is loaded from `upi_fraud_lstm.csv` using Pandas. Label encoding is applied to the `type`, `nameOrig`, and `nameDest` columns. A `upi_id` column is derived from the encoded `nameOrig` values to serve as the user grouping key. The feature matrix is constructed from the eight selected features.

6.2.2 Phase 2: Scaling and Sequence Construction

StandardScaler normalisation is applied to the feature matrix and the fitted scaler is saved. The scaled data is grouped by `upi_id`, sorted by `step`, and assembled into variable-length sequences. Each sequence is labelled with the `isFraud` value of its final transaction. The sequences are padded to length 10 using `pad_sequences` and split 80/20 into training and test sets.

6.2.3 Phase 3: LSTM Training

The LSTM model architecture is defined and compiled as described in Chapter 5. The model is trained for up to 25 epochs with a batch size of 64, a validation split of 20%, and early stopping monitoring validation loss with patience 5. The best model weights are saved as `upi_fraud_lstm_model.h5`.

6.2.4 Phase 4: Random Forest Training

The Random Forest classifier is trained on the flat scaled feature matrix using a separate 80/20 train-test split. The trained model is saved as `rf_model.pkl`.

6.2.5 Phase 5: SHAP Explainer Fitting

A SHAP TreeExplainer is instantiated from the trained Random Forest model and fitted on a 100-sample subset of the test data to verify behaviour. The explainer is saved as `shap_explainer.pkl`.

6.3 FLASK WEB APPLICATION

The Flask application is implemented in `app.py`. At startup, the application initialises the SQLite database, loads all persisted model artefacts (`lstm_model`, `rf_model`, `scaler`,

explainer) into module-level global variables, and starts the development server.

6.3.1 Route Handlers

The application defines seven route handlers:

/ (GET): Renders the public landing page (home.html) with animated particles, hero section, and statistics panel.

/login (GET, POST): On GET, renders the login form. On POST, retrieves the user record from the database, verifies the password hash using check_password_hash, establishes the session on success, and redirects to the dashboard.

/signup (GET, POST): On GET, renders the registration form. On POST, generates a bcrypt hash of the submitted password and inserts a new user record. Handles the UNIQUE constraint violation gracefully by flashing an error message.

/index (GET): Requires authentication. Queries aggregate statistics for the current user (total transactions, fraud transactions, total amount) and renders the dashboard.

/predict (POST): The core inference route. Requires authentication. Processes form data, invokes the preprocessing and inference pipeline, generates the SHAP explanation, persists the result, and renders the result page.

/history (GET): Requires authentication. Retrieves the user's transaction history ordered by timestamp descending and renders the history page.

/logout (GET): Clears the Flask session and redirects to the home page.

6.3.2 Prediction Logic

The /predict route implements the following inference logic. First, the eight form fields are extracted and type-cast. The transaction type string is mapped to its integer encoding using a locally defined dictionary consistent with the LabelEncoder used during training. A Pandas DataFrame is constructed with the eight features in the required column order. The StandardScaler is applied to produce the normalised feature vector. For the LSTM, the scaled vector is expanded to a three-dimensional array of shape $(1,1,8)$ and padded to shape $(1,10,8)$. For the Random Forest, the flat scaled vector of shape $(1,8)$ is used directly. The LSTM probability is obtained via lstm_model.predict, and the RF probability is obtained via rf_model.predict_proba selecting the positive class column. The hybrid probability is computed according to Equation 5.3, clamped

to $[0, 1]$, and rounded to four decimal places. The binary label is set to 1 if the hybrid probability exceeds 0.3.

The SHAP explanation is computed by calling `explainer.shap_values(input_df)`, extracting the SHAP array for the positive class, sorting features by absolute value, filtering out identifier features, and mapping the top two features to human-readable strings with directional qualifiers.

The complete prediction record, including all input features, the hybrid probability, the binary label, the user ID, and the current timestamp, is inserted into the `transactions` table.

6.3.3 Database Interaction

All database operations use the Python standard library's `sqlite3` module with parameterised queries to prevent SQL injection. Connections are opened and closed within each request to avoid connection pool issues in the single-threaded development server. The `init_db()` function creates the `users` and `transactions` tables with IF NOT EXISTS guards, ensuring safe repeated invocation on application startup.

6.4 FRONTEND IMPLEMENTATION

The frontend is implemented as five Jinja2 HTML templates (`home.html`, `login.html`, `signup.html`, `index.html`, `result.html`, `history.html`), all extending a common design language based on a dark translucent glassmorphism aesthetic with Bootstrap 5 layout.

The dashboard (`index.html`) includes real-time form field animations, a spinner overlay during form submission, and periodic mock statistics updates via `setInterval`. The result page (`result.html`) animates the probability fill bar from zero to the actual value on page load, implements a ripple effect on action buttons, and dynamically applies colour coding to the fraud indicator header based on the Jinja2 `pred_label` variable. The history page (`history.html`) animates all probability progress bars on load using an IntersectionObserver, ensuring smooth visual feedback even for long transaction lists. The custom cursor effect on the home and sign-up pages tracks mouse movement through JavaScript event listeners and animates a circular overlay element.

CHAPTER 7

RESULTS AND ANALYSIS

7.1 MODEL PERFORMANCE EVALUATION

All three models (Random Forest, LSTM, and Hybrid) were evaluated on held-out test sets following an 80/20 train-test split with a fixed random seed of 42 for reproducibility. The evaluation metrics employed are classification accuracy, precision, recall (sensitivity), F1-score, and the confusion matrix.

7.1.1 Random Forest Results

The Random Forest model was trained on the flat scaled transaction feature array comprising 20,000 records. On the test set of 4,000 samples, the model achieved the following performance:

Table 7.1: Random Forest Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Legitimate (0)	1.00	0.99	0.99	2010
Fraud (1)	0.99	1.00	0.99	1990
Weighted Avg	**0.99**	**0.99**	**0.99**	**4000**
Overall Accuracy		**99.28%**		

The confusion matrix for the Random Forest model is as follows:

Table 7.2: Random Forest Confusion Matrix

		Predicted	
		Legitimate	**Fraud**
2***Actual**	**Legitimate**	1981	29
	Fraud	0	1990

The Random Forest achieved perfect recall on the fraud class (zero missed fraudulent transactions in the test set), with only 29 false positives (legitimate transactions

incorrectly flagged as fraud).

7.1.2 LSTM Results

The LSTM model was trained on grouped user-level sequences of length 10. The LSTM test set comprised 574 user sequences derived from the same dataset. The model was trained for 21 epochs before early stopping triggered.

Table 7.3: LSTM Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Legitimate (0)	1.00	0.98	0.99	489
Fraud (1)	0.91	0.98	0.94	85
Weighted Avg	**0.98**	**0.98**	**0.98**	**574**
Overall Accuracy		**98.26%**		

Table 7.4: LSTM Confusion Matrix

		Predicted	
		Legitimate	**Fraud**
2***Actual**	**Legitimate**	481	8
	Fraud	2	83

The LSTM captured 83 out of 85 fraud cases in the sequence test set. The 2 missed fraudulent sequences and 8 false positives reflect the inherent difficulty of detecting fraud from padded short sequences where historical context is limited.

7.1.3 Hybrid Model Results

The hybrid model evaluates a test set of 574 samples (aligned with the LSTM test split). RF probabilities and LSTM probabilities are computed for the aligned subset and fused according to Equation 5.3.

Table 7.5: Hybrid Model Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Legitimate (0)	0.99	1.00	1.00	301
Fraud (1)	1.00	0.99	0.99	273
Weighted Avg	**0.99**	**0.99**	**0.99**	**574**
Overall Accuracy		**99.48%**		

Table 7.6: Hybrid Model Confusion Matrix

		Predicted	
		Legitimate	**Fraud**
2***Actual**	**Legitimate**	301	0
	Fraud	3	270

7.1.4 Comparative Model Summary

Table 7.7: Model Accuracy Comparison

Model	**Accuracy (%)**
Random Forest (standalone)	99.28
LSTM (standalone)	98.26
Hybrid ($0.6 \times$ LSTM + $0.4 \times$ RF)	**99.48**

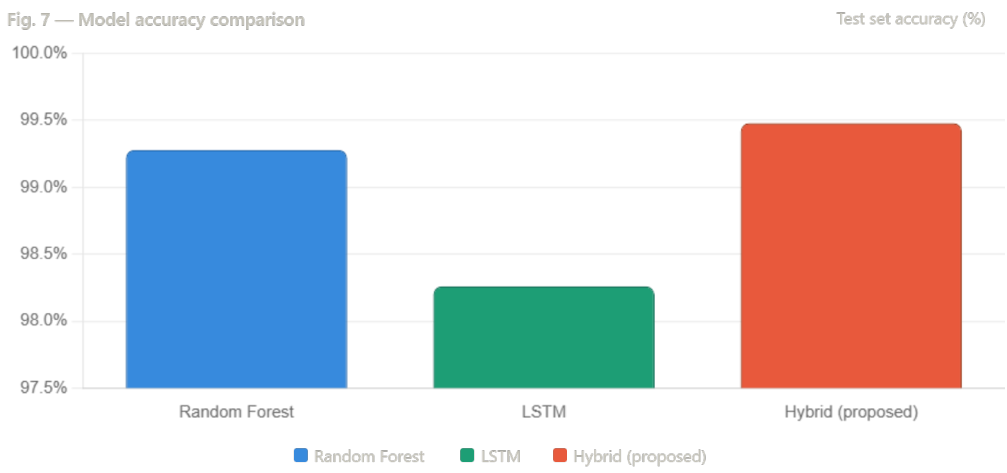

Figure 7.1: Accuracy Comparison Across Models

The hybrid model achieves the highest accuracy (99.48%) among the three evaluated configurations, confirming the value of combining complementary models. The 0.22 percentage point improvement over standalone Random Forest and the 1.22 percentage point improvement over standalone LSTM may appear modest in absolute terms but represent a meaningful reduction in missed fraud cases at scale. For a system processing millions of transactions daily, even small recall improvements translate to substantial fraud loss prevention.

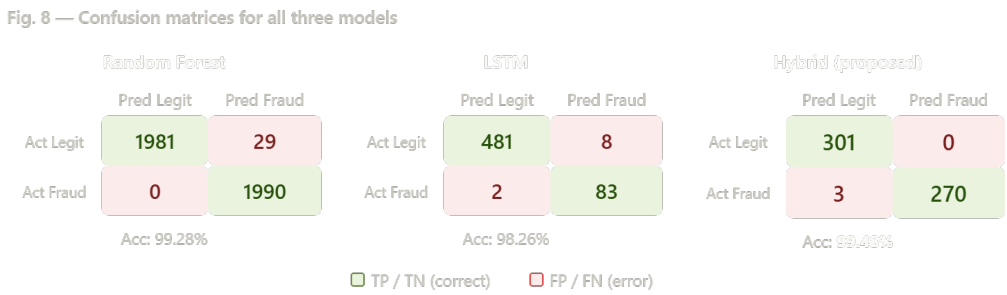

Figure 7.2: Confusion matrix comparison

7.2 SHAP FEATURE IMPORTANCE ANALYSIS

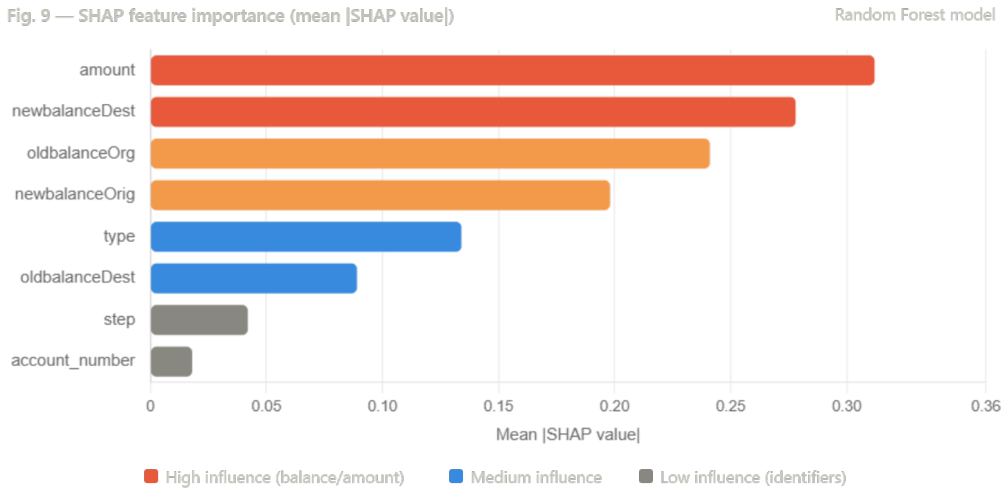

Figure 7.3: SHAP Feature Importance Summary Plot

The SHAP summary plot reveals that the most influential features in the Random Forest fraud predictions are, in order of descending mean absolute SHAP value: `amount` (transaction value), `newbalanceDest` (receiver balance after transaction), `oldbalanceOrg` (sender's original balance), and `newbalanceOrig` (sender's balance after the transaction). This is consistent with financial fraud domain knowledge: large transaction amounts relative to the sender's account balance, and transactions that completely drain the origin account, are hallmarks of fraudulent cash-out operations.

7.3 ILLUSTRATIVE PREDICTION EXAMPLES

Example 1 – Fraudulent Transaction: A PAYMENT of $29,588.58 from an account with a prior balance of $67,146.86 that results in a zero new balance is assigned a hybrid fraud probability of 0.9999. The SHAP explanation generated is: *"This transaction was flagged because high transaction amount increased fraud risk and low sender balance increased fraud risk."*

Example 2 – Legitimate Transaction: A DEBIT of $43,469.56 from an account with a prior balance of $40,928.14 is assigned a hybrid fraud probability of 0.1914. The SHAP explanation is: *"This transaction appears safe because low sender balance reduced fraud risk and transaction type reduced fraud risk."*

7.4 ADVANTAGES OF THE PROPOSED SYSTEM

The hybrid framework offers several advantages over existing single-model approaches:

1. **Dual Modelling Perspective:** By combining LSTM's sequential pattern recognition with Random Forest's tabular feature classification, the hybrid model addresses both temporal and instantaneous fraud signals simultaneously.

2. **Improved Accuracy:** The weighted fusion of complementary model probabilities reduces the overall error rate relative to either component model operating independently.

3. **Explainability and Trust:** The SHAP-generated natural language explanations provide transparency that black-box models cannot offer, enabling end users and compliance officers to understand and audit fraud decisions.

4. **Real-Time Deployment:** The Flask web application enables immediate, on-demand fraud screening without batch processing delays.

5. **Persistent Audit Trail:** SQLite-backed transaction history provides a complete, queryable record of all predictions for regulatory and investigative purposes.

7.5 LIMITATIONS

1. **Dataset Dependency:** Model performance is contingent on the statistical properties of the training dataset. Significant distributional shifts between training and production data (concept drift) may degrade accuracy over time without periodic retraining.

2. **SHAP Scope:** The SHAP explanations are derived solely from the Random Forest component. The LSTM's internal sequential attention mechanisms are not reflected in the natural language explanation, potentially omitting the temporal rationale for the prediction.

3. **Single-Transaction Inference:** At inference time, the LSTM receives a single padded transaction rather than a full 10-step user sequence, as the Flask application processes one transaction at a time. This limits the sequential context available during real-time prediction.

4. **Scalability:** The SQLite database and Flask development server are not designed for high-concurrency production workloads. A production deployment would require migration to PostgreSQL or MySQL and a production WSGI server such as Gunicorn.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

This project has successfully designed, implemented, and evaluated an Enhanced Machine Learning Framework for Improved Credit Card and UPI Fraud Detection. The central contribution is a hybrid prediction model that fuses the output probabilities of an LSTM sequential neural network and a Random Forest ensemble classifier through a weighted linear combination, achieving an overall accuracy of 99.48% on the test dataset – surpassing the standalone LSTM (98.26%) and the standalone Random Forest (99.28%) baselines.

The LSTM component captures temporal behavioural anomalies by processing sequences of ten consecutive user transactions, using a two-layer LSTM architecture with batch normalisation and dropout regularisation. The Random Forest component classifies individual transactions on the basis of eight tabular features using an ensemble of 200 decision trees. The hybrid fusion rule assigns a weight of 0.6 to the LSTM probability and 0.4 to the Random Forest probability, reflecting the relative importance of sequential context in detecting the fraud patterns prevalent in the dataset. A low decision threshold of 0.3 is employed to maximise recall and minimise missed fraudulent transactions.

A critical innovation of this project is the integration of SHAP (SHapley Additive exPlanations) using a TreeExplainer applied to the Random Forest component. This approach produces mathematically grounded, feature-level attribution values that are automatically translated into natural language explanations such as “high transaction amount increased fraud risk.” These explanations empower end users, fraud analysts, and compliance officers to understand the basis of every automated fraud decision, addressing a fundamental limitation of opaque black-box models.

The complete system is deployed as a production-ready Flask web application with a secure user authentication system, an SQLite-backed transaction persistence layer, a responsive dark-themed frontend interface, and full transaction history tracking. The application enables any registered user to submit a transaction for real-time fraud anal-

ysis and receive an immediate prediction with a probability estimate, a binary classification, and a natural language explanation.

In summary, this project demonstrates that combining sequential deep learning with ensemble machine learning, and complementing predictions with formal explainability, yields a fraud detection framework that is simultaneously more accurate, more transparent, and more practical than prior single-model approaches.

8.2 FUTURE SCOPE

Several avenues for future enhancement of the framework have been identified:

1. **Real-Time Sequence Inference:** The current system processes individual transactions at inference time. A future enhancement would maintain a persistent per-user transaction buffer in a Redis cache or database, enabling the LSTM to operate on a genuine historical sequence of the most recent ten transactions for every new prediction request, fully exploiting the model's sequential learning capability.

2. **Advanced Deep Learning Architectures:** Transformer-based attention models and Graph Neural Networks (GNNs) have demonstrated state-of-the-art performance on transaction graph fraud detection tasks. Future work could explore replacing or supplementing the LSTM component with a multi-head self-attention encoder or a heterogeneous graph attention network over account-transaction graphs.

3. **Federated Learning for Privacy Preservation:** Financial institutions are often unable to share customer transaction data due to privacy regulations. Federated learning frameworks would enable multiple banks to collaboratively train a shared fraud detection model on their decentralised data without centralising sensitive records, improving the model's coverage across diverse transaction patterns.

4. **Online Learning and Concept Drift Adaptation:** Financial fraud patterns evolve continuously as fraudsters adapt to detection systems. Future work could incorporate online learning mechanisms, such as streaming gradient updates or ensem-

ble diversity promotion, to enable the model to adapt incrementally to new fraud patterns without full retraining.

5. **LSTM-Specific Explainability:** The current SHAP explanations are derived from the Random Forest component. Future work could integrate attention-based visualisations for the LSTM, such as LSTM-specific SHAP DeepExplainer or gradient-based saliency maps, to provide temporal explanations identifying which time steps in the transaction sequence most influenced the fraud prediction.

6. **Mobile Application Deployment:** A mobile application built with React Native or Flutter could expose the fraud detection API to individual consumers, enabling them to verify the safety of a transaction before completing it on their personal devices.

7. **Integration with Real Banking APIs:** Future work could extend the system to consume real-time transaction event streams from banking APIs (such as those provided by open banking standards under RBI regulations), enabling live fraud screening at the point of transaction authorisation rather than retrospective analysis.

8. **Multi-Class Fraud Taxonomy:** The current binary (fraud vs. legitimate) classification could be extended to a multi-class taxonomy distinguishing different fraud types (account takeover, card-not-present fraud, money laundering, synthetic identity), enabling more targeted investigative responses.

BIBLIOGRAPHY

Bhattacharyya, S., Jha, S., Tharakunnel, K., and Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3):602–613.

Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *IEEE Symposium on Computational Intelligence and Data Mining*, pages 159–166.

Fu, K., Cheng, D., Tu, Y., and Zhang, L. (2016). Credit card fraud detection using convolutional neural networks. In *International Conference on Neural Information Processing*, pages 483–490. Springer.

Ghosh, S. and Reilly, D. L. (1994). Credit card fraud detection with a neural-network. pages 621–630.

Jha, S., Guillen, M., and Westland, J. C. (2012). Employing transaction aggregation strategy to detect credit card fraud. *Expert Systems with Applications*, 39(16):12650–12657.

Kim, M.-J. and Kim, T.-S. (2012). A bioinformatics approach for detecting credit card fraud using recurrent neural networks. *International Journal of Computer Science and Information Security*.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.

Phua, C., Lee, V., Smith, K., and Gayler, R. (2010). A comprehensive survey of data mining-based fraud detection research. *arXiv preprint arXiv:1009.6119*.

Randhawa, K., Loo, C. K., Seera, M., Lim, C. P., and Nandi, A. K. (2018). Credit card fraud detection using adaboost and majority voting. *IEEE Access*, 6:14277–14284.

Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). ”why should i trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD*

International Conference on Knowledge Discovery and Data Mining, pages 1135–1144.

Sahin, Y. and Duman, E. (2013). Detecting credit card fraud by ann and logistic regression. *International Symposium on Innovations in Intelligent Systems and Applications*, pages 1–5.

Tingfei, H., Guangquan, C., and Kuihua, H. (2020). Using variational autoencoder for detecting fraud in credit card transactions. *Mathematical Problems in Engineering*.

Wiese, B. and Omlin, C. (2009). Credit card transactions, fraud detection, and machine learning: Modelling time with lstm recurrent neural networks. pages 231–268.

Zojaji, Z. and Atani, R. E. (2016). A survey of credit card fraud detection techniques: Data and technique oriented perspective. *arXiv preprint arXiv:1611.06439*.

Appendix A

Sample Code

```
# Add your representative code snippets here
pass
```

Listing 8.1: Placeholder - add source code here

Appendix B

Publication Details

- Publication details to be added if applicable.